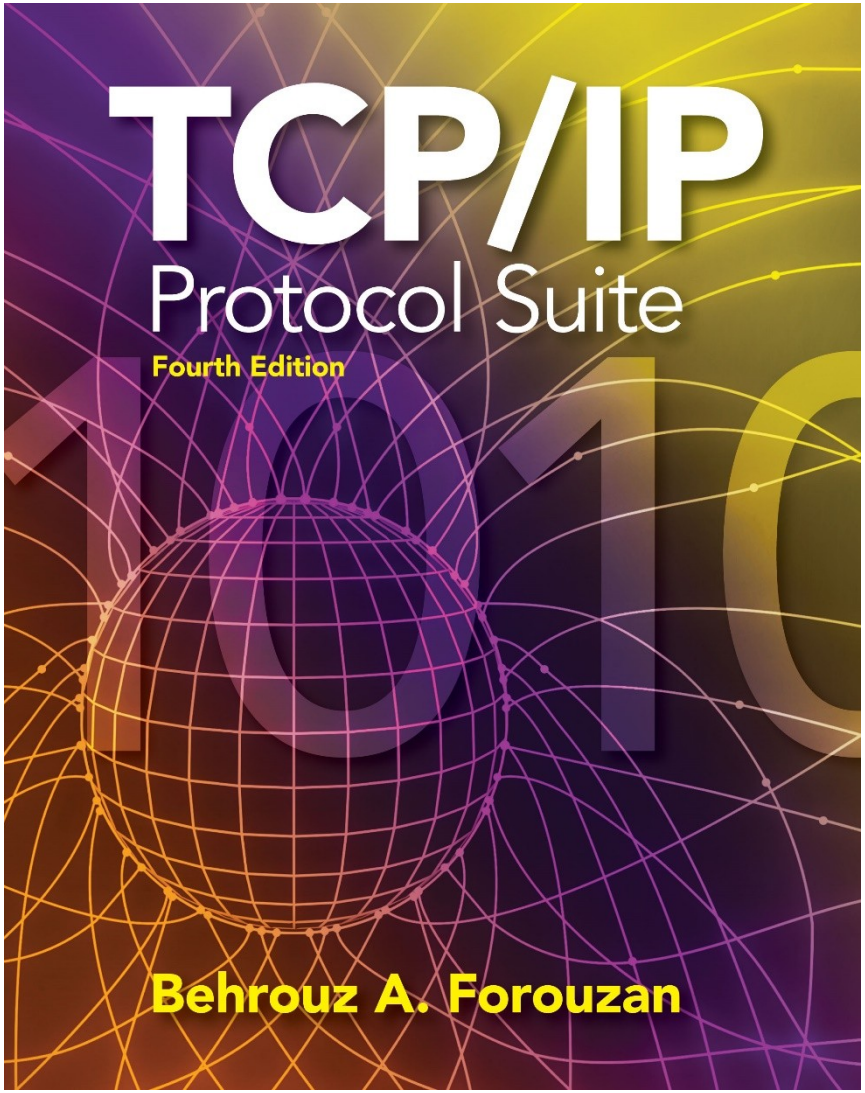


↳ Connectionless Protocol

OBJECTIVES:

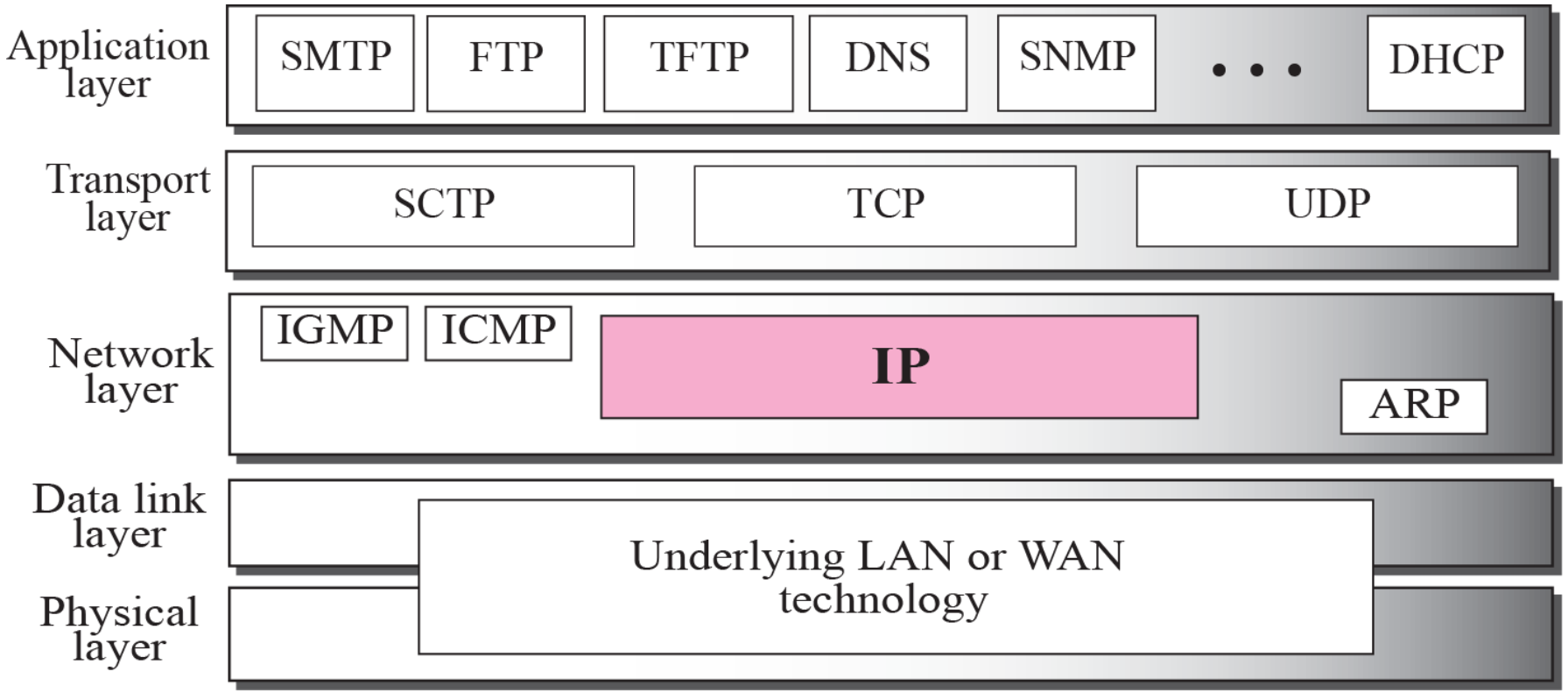
! " #
$ $ % #

Chapter Outline

The Internet Protocol (IP) is the transmission mechanism used by the TCP/IP protocols at the network layer.

&

'


Application layer
SMTP
FTP
TFTP
DNS
SNMP
. . .
DHCP
Transport layer
SCTP
TCP
UDP
Network layer
IGMP
ICMP
IP
ARP
Data link layer
Underlying LAN or WAN technology
Physical layer

! "#

Packets in the network (internet) layer are called . A datagram is a variable-length packet consisting of two parts: header and data. The header is 20 to 60 bytes in length and contains information essential to routing and delivery. It is customary in TCP/IP to show the header in 4-byte sections. A brief description of each field is in order.

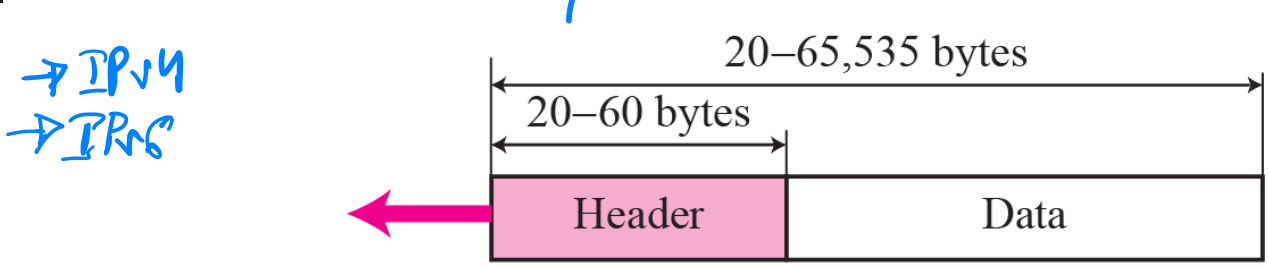
(

'

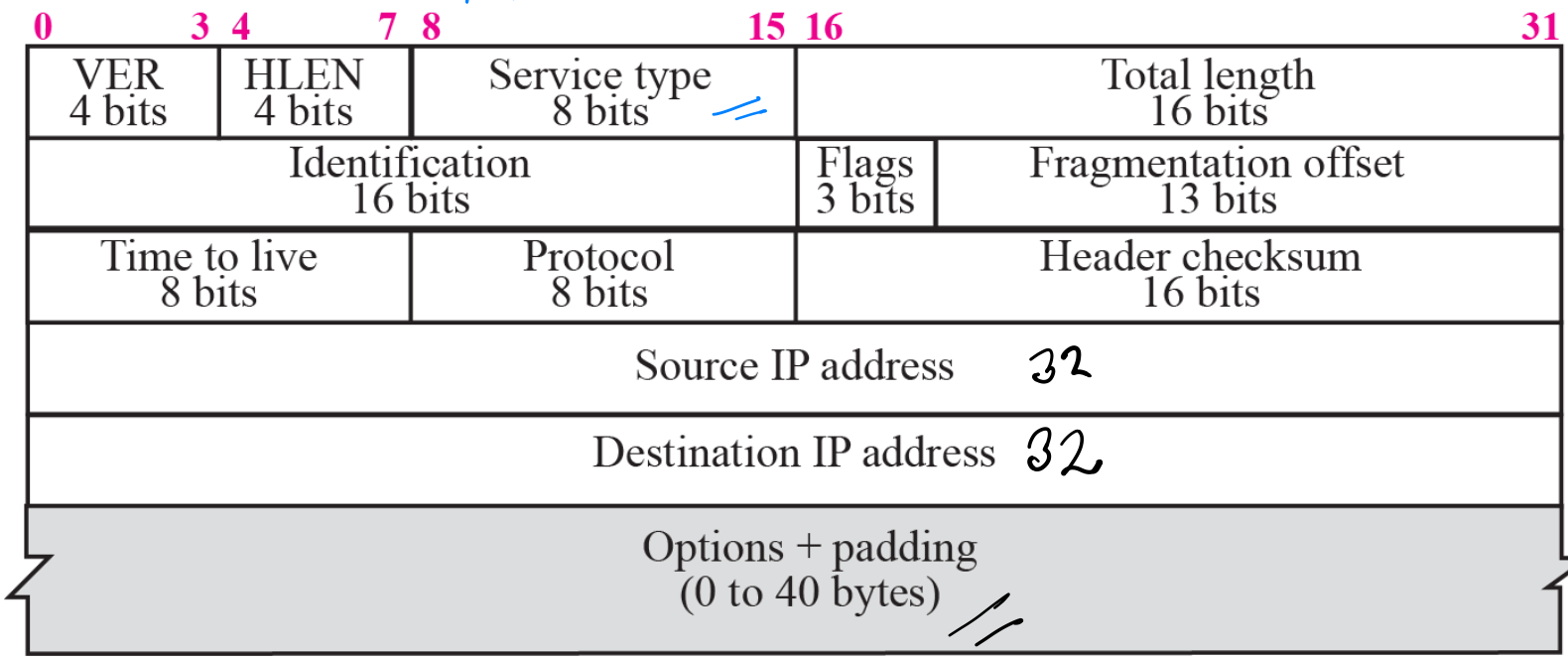
Datagram ← Header Size = 20 – 60 B

↖ Payload = 0 – 65515 B

↳ $2^{16} - 1 = 65535$



a. IP datagram



b. Header format

- **Version (VER).** This 4-bit field defines the version of the IP protocol. Currently the version is 4. However, version 6 (or IPv6) may totally replace version 4 in the future. This field tells the IP software running in the processing machine that the datagram has the format of version 4. All fields must be interpreted as specified in the fourth version of the protocol. If the machine is using some other version of IP, the datagram is discarded rather than interpreted incorrectly.

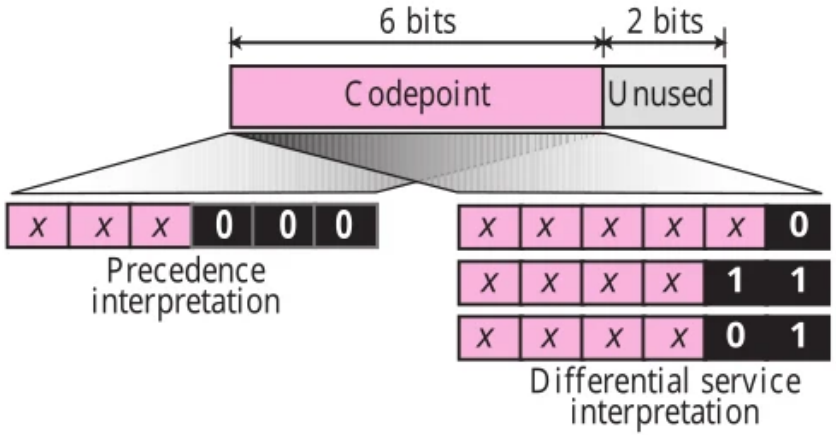
- **Header length (HLEN).** This 4-bit field defines the total length of the datagram header in 4-byte words. This field is needed because the length of the header is variable (between 20 and 60 bytes). When there are no options, the **header length** is 20 bytes, and the value of this field is 5 ($5 \times 4 = 20$). When the option field is at its maximum size, the value of this field is 15 ($15 \times 4 = 60$).

- **Service type.** In the original design of IP header, this field was referred to as **type of service (TOS)**, which defined how the datagram should be handled. Part of the field was used to define the precedence of the datagram; the rest defined the type of service (low delay, high throughput, and so on). IETF has changed the interpretation of this 8-bit field. This field now defines a set of **differentiated services.** The new interpretation is shown in Figure 7.3.

  In this interpretation, the first 6 bits make up the **codepoint** subfield and the last 2 bits are not used. The codepoint subfield can be used in two different ways.

  a. When the 3 right-most bits are 0s, the 3 left-most bits are interpreted the same as the precedence bits in the service type interpretation. In other words, it is compatible with the old interpretation. The precedence defines the eight-level

Figure 7.3 *Service type*



priority of the datagram (0 to 7) in issues such as congestion. If a router is congested and needs to discard some datagrams, those datagrams with lowest precedence are discarded first. Some datagrams in the Internet are more important than the others. For example, a datagram used for network management is much more urgent and important than a datagram containing optional information for a group.

b. When the 3 right-most bits are not all 0s, the 6 bits define 56 (64 – 8) services based on the priority assignment by the Internet or local authorities according to Table 7.1. The first category contains 24 service types; the second and the third each contain 16. The first category is assigned by the Internet authorities (IETF). The second category can be used by local authorities (organizations). The third category is temporary and can be used for experimental purposes. Note that these assignments have not yet been finalized.

Table 7.1 *Values for codepoints*

Category	*Codepoint*	*Assigning Authority*
1	XXXXX0	Internet
2	XXXX11	Local
3	XXXX01	Temporary or experimental

- **Total length.** This is a 16-bit field that defines the total length (header plus data) of the IP datagram in bytes. To find the length of the data coming from the upper layer, subtract the header length from the total length. The header length can be found by multiplying the value in the HLEN field by four.

$$\text{Length of data} = \text{total length} - \text{header length}$$

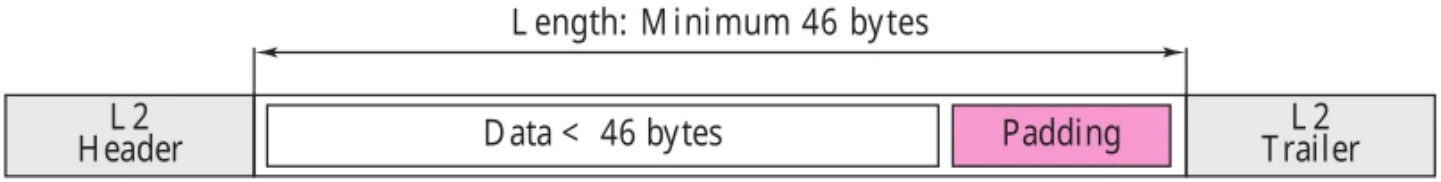
Since the field length is 16 bits, the total length of the IP datagram is limited to 65,535 ($2^{16} - 1$) bytes, of which 20 to 60 bytes are the header and the rest is data from the upper layer.

The *total length* field defines the total length of the datagram including the header.

Though a size of 65,535 bytes might seem large, the size of the IP datagram may increase in the near future as the underlying technologies allow even more throughput (more bandwidth). When we discuss fragmentation in the next section, we will see that some physical networks are not able to encapsulate a datagram of 65,535 bytes in their frames. The datagram must be fragmented to be able to pass through those networks.

One may ask why we need this field anyway. When a machine (router or host) receives a frame, it drops the header and the trailer leaving the datagram. Why include an extra field that is not needed? The answer is that in many cases we really do not need the value in this field. However, there are occasions in which the datagram is not the only thing encapsulated in a frame; it may be that padding has been added. For example, the Ethernet protocol has a minimum and maximum restriction on the size of data that can be encapsulated in a frame (46 to 1500 bytes). If the size of an IP datagram is less than 46 bytes, some padding will be added to meet this requirement. In this case, when a machine decapsulates the datagram, it needs to check the total length field to determine how much is really data and how much is padding (see Figure 7.4).

Figure 7.4 *Encapsulation of a small datagram in an Ethernet frame*



- **Identification.** This field is used in fragmentation (discussed in the next section).
- **Flags.** This field is used in fragmentation (discussed in the next section).
- **Fragmentation offset.** This field is used in fragmentation (discussed in the next section).
- **Time to live.** A datagram has a limited lifetime in its travel through an internet. This field was originally designed to hold a timestamp, which was decremented by each visited router. The datagram was discarded when the value became zero. However, for this scheme, all the machines must have synchronized clocks and must know how long it takes for a datagram to go from one machine to another. Today, this field is mostly used to control the maximum number of hops (routers) visited by the datagram. When a source host sends the datagram, it stores a number in this field. This value is approximately two times the maximum number of routes between any two hosts. Each router that processes the datagram decrements this number by one. If this value, after being decremented, is zero, the router discards the datagram.

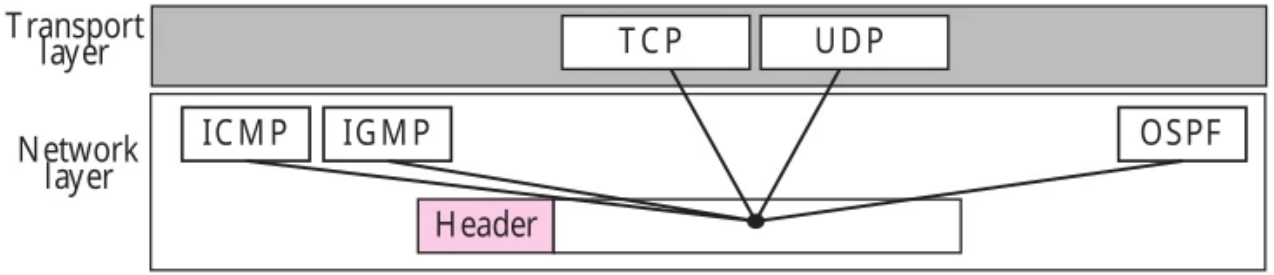
  This field is needed because routing tables in the Internet can become corrupted. A datagram may travel between two or more routers for a long time without ever getting delivered to the destination host. This field limits the lifetime of a datagram.

  Another use of this field is to intentionally limit the journey of the packet. For example, if the source wants to confine the packet to the local network, it can store

1 in this field. When the packet arrives at the first router, this value is decremented to 0, and the datagram is discarded.

- **Protocol.** This 8-bit field defines the higher-level protocol that uses the services of the IP layer. An IP datagram can encapsulate data from several higher level protocols such as TCP, UDP, ICMP, and IGMP. This field specifies the final destination protocol to which the IP datagram should be delivered. In other words, since the IP protocol multiplexes and demultiplexes data from different higher-level protocols, the value of this field helps in the demultiplexing process when the datagram arrives at its final destination (see Figure 7.5).

Figure 7.5 *Multiplexing*



Some of the value of this field for different higher-level protocols is shown in Table 7.2.

Table 7.2 *Protocols*

Value	*Protocol*	*Value*	*Protocol*
1	ICMP	17	UDP
2	IGMP	89	OSPF
6	TCP		

- **Checksum.** The checksum concept and its calculation are discussed later in this chapter.
- **Source address.** This 32-bit field defines the IP address of the source. This field must remain unchanged during the time the IP datagram travels from the source host to the destination host.
- **Destination address.** This 32-bit field defines the IP address of the destination. This field must remain unchanged during the time the IP datagram travels from the source host to the destination host.

Example 7.1

An IP packet has arrived with the first 8 bits as shown:

```
01000010
```

The receiver discards the packet. Why?

Solution

There is an error in this packet. The 4 left-most bits (0100) show the version, which is correct. The next 4 bits (0010) show the wrong header length ($2 \times 4 = 8$). The minimum number of bytes in the header must be 20. The packet has been corrupted in transmission.

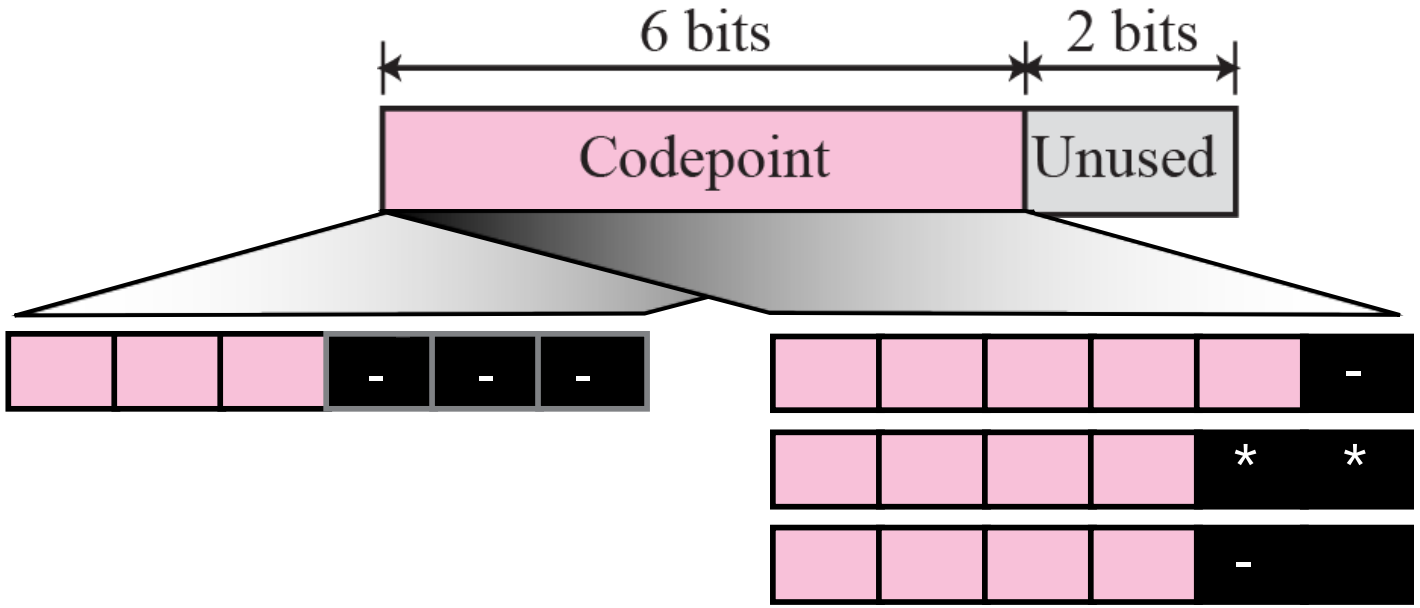
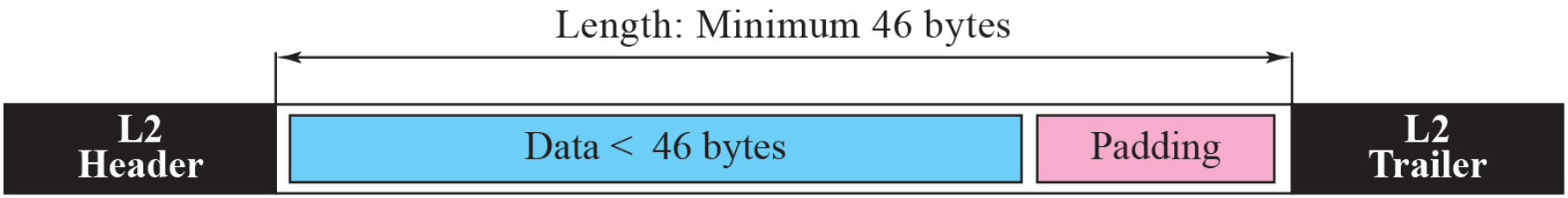

6 bits
2 bits
Codepoint
Unused
-
-
-
-
* *
-

Table 7.1 *Values for codepoints*

Category	*Codepoint*	*Assigning Authority*
1	XXXXX0	Internet
2	XXXX11	Local
3	XXXX01	Temporary or experimental


Length: Minimum 46 bytes
L2 Header
Data < 46 bytes
Padding
L2 Trailer

( ).

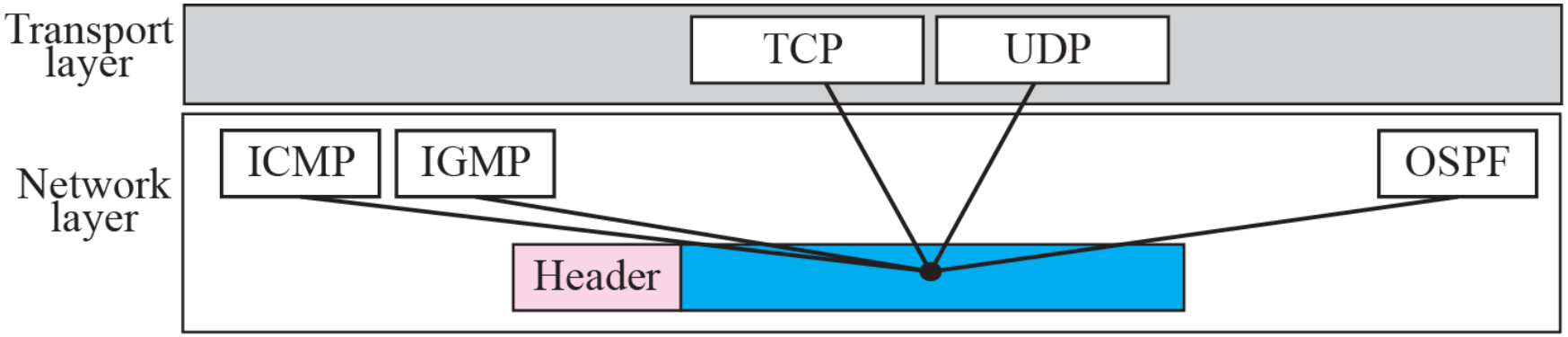

Transport layer
TCP
UDP
Network layer
ICMP
IGMP
OSPF
Header

Table 7.2 *Protocols*

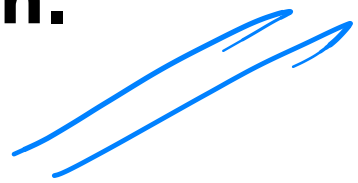
Value	*Protocol*	*Value*	*Protocol*
1	ICMP	17	UDP
2	IGMP	89	OSPF
6	TCP		

An IP packet has arrived with the first 8 bits as shown:

01000010

The receiver discards the packet. Why?

There is an error in this packet. The 4 left-most bits (0100) show the version, which is correct. The next 4 bits (0010) show the wrong header length (2 × 4 = 8). The minimum number of bytes in the header must be 20. The packet has been corrupted in transmission.



In an IP packet, the value of HLEN is 1000 in binary. How many bytes of options are being carried by this packet?

The HLEN value is 8, which means the total number of bytes in the header is 8 × 4 or 32 bytes. The first 20 bytes are the base header, the next 12 bytes are the options.

In an IP packet, the value of HLEN is 5_{16} and the value of the total length field is 0028_{16}. How many bytes of data are being carried by this packet?

The HLEN value is 5, which means the total number of bytes in the header is 5 × 4 or 20 bytes (no options). The total length is 40 bytes, which means the packet is carrying 20 bytes of data (40 − 20).

An IP packet has arrived with the first few hexadecimal digits as shown below:

```
45000028000100000102 . . .
```

How many hops can this packet travel before being dropped? The data belong to what upper layer protocol?

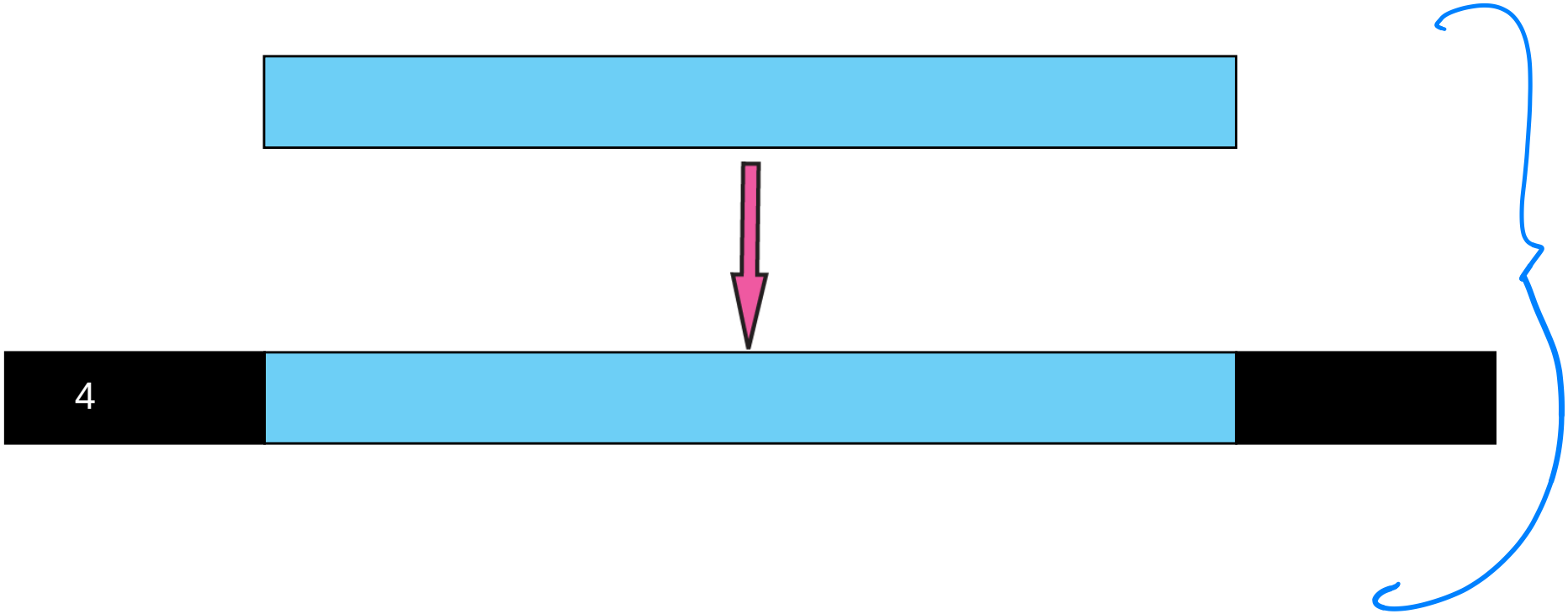
To find the time-to-live field, we skip 8 bytes (16 hexadecimal digits). The time-to-live field is the ninth byte, which is 01. This means the packet can travel only one hop. The protocol field is the next byte (02), which means that the upper layer protocol is IGMP (see Table 7.2)

$ % !"&

A datagram can travel through different networks. Each router decapsulates the IP datagram from the frame it receives, processes it, and then encapsulates it in another frame. The format and size of the received frame depend on the protocol used by the physical network through which the frame has just traveled. The format and size of the sent frame depend on the protocol used by the physical network through which the frame is going to travel.

! 0 1! 02

(& (


4

The value of the MTU differs from one physical network protocol to another. For example, the value for the Ethernet LAN is 1500 bytes, for FDDI LAN is 4352 bytes, and for PPP is 296 bytes.

In order to make the IP protocol independent of the physical network, the designers decided to make the maximum length of the IP datagram equal to 65,535 bytes. This makes transmission more efficient if we use a protocol with an MTU of this size. However, for other physical networks, we must divide the datagram to make it possible to pass through these networks. This is called **fragmentation.**

The source usually does not fragment the IP packet. The transport layer will instead segment the data into a size that can be accommodated by IP and the data link layer in use.

When a datagram is fragmented, each fragment has its own header with most of the fields repeated, but some changed. A fragmented datagram may itself be fragmented if it encounters a network with an even smaller MTU. In other words, a datagram can be fragmented several times before it reaches the final destination.

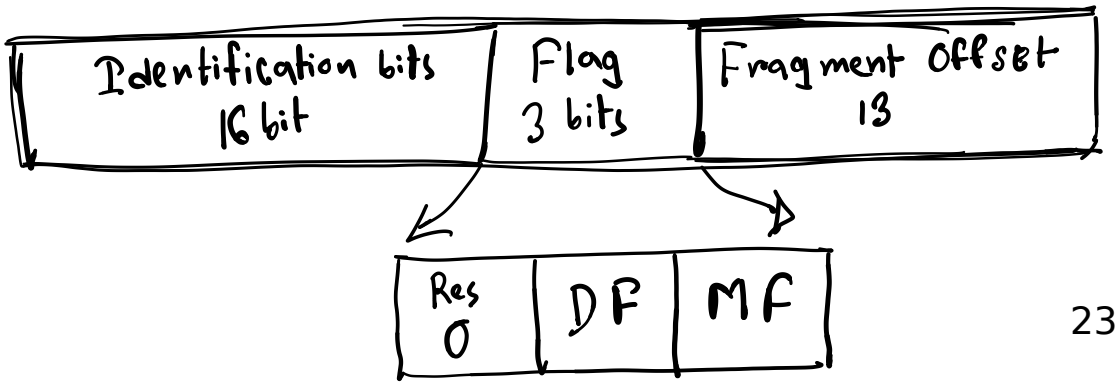
A datagram can be fragmented by the source host or any router in the path. The reassembly of the datagram, however, is done only by the destination host because each fragment becomes an independent datagram. Whereas the fragmented datagram can travel through different routes, and we can never control or guarantee which route a fragmented datagram may take, all of the fragments belonging to the same datagram should finally arrive at the destination host. So it is logical to do the reassembly at the final destination. An even stronger objection for reassembling packets during the transmission is the loss of efficiency it incurs.

When a datagram is fragmented, required parts of the header must be copied by all fragments. The option field may or may not be copied as we will see in the next section. The host or router that fragments a datagram must change the values of three fields: flags, fragmentation offset, and total length. The rest of the fields must be copied. Of course, the value of the checksum must be recalculated regardless of fragmentation.

Note

Only data in a datagram is fragmented.

Fragmentation =



4️⃣ Key Header Fields Used in Fragmentation

1️⃣ Identification (16 bits)

- Same value for all fragments of a packet
- Used to group fragments during reassembly

2️⃣ Flags (3 bits)

Bit	Name	Meaning
0	Reserved	Always 0
1	DF	Don't Fragment
2	MF	More Fragments

📌 MF = 1 → more fragments coming
📌 MF = 0 → last fragment

3️⃣ Fragment Offset (13 bits)

- Position of fragment's data within original packet
- Measured in 8-byte units

📌 This is why fragment sizes (except last) must be multiples of 8

5️⃣ How Fragmentation Works (Step-by-Step Example)

📦 Original Datagram

- Total size = 4000 bytes
- Header = 20 bytes
- Data = 3980 bytes
- MTU = 1500 bytes

✂️ Step 1: Determine max data per fragment

yaml Copy code

```
MTU − Header = 1500 − 20 = 1480 bytes
```

1480 is divisible by 8 ✔️

✂️ Step 2: Fragment the data

Fragment	Data (bytes)	Total Length	Offset	MF
1	1480	1500	0	1
2	1480	1500	185	1
3	1020	1040	370	0

📌 Offset = (bytes sent before) ÷ 8
Example: 1480 ÷ 8 = 185

6️⃣ Reassembly at Destination

At the destination host:

- Fragments are grouped using:
  - Source IP
  - Destination IP
  - Protocol
  - Identification field
- Ordered using Fragment Offset
- Last fragment detected using MF = 0

📌 If any fragment is missing, the entire packet is discarded

7️⃣ DF (Don't Fragment) Flag

- If DF = 1 and fragmentation is needed:
  - Router drops packet
  - Sends ICMP "Fragmentation Needed" message back

📌 Used by Path MTU Discovery (PMTUD)

8️⃣ Problems with Fragmentation

❌ Inefficient
❌ Increased overhead (each fragment has a header)
❌ If one fragment is lost → whole packet lost
❌ Firewalls may block fragments
❌ Performance issues

📌 This is why modern networks try to avoid fragmentation

9️⃣ Fragmentation vs Segmentation (Very Important)

Aspect	Fragmentation	Segmentation
Layer	Network (IP)	Transport (TCP)
Happens due to	MTU limit	MSS / application size
Who does it	Router / Host	TCP
Reliability	No	Yes (TCP)

🔑 One-Line Exam Answers

What is IP fragmentation?

> Breaking an IP packet into smaller packets to fit MTU limits.

Where is reassembly done?

> Only at the destination host.

Which fields control fragmentation?

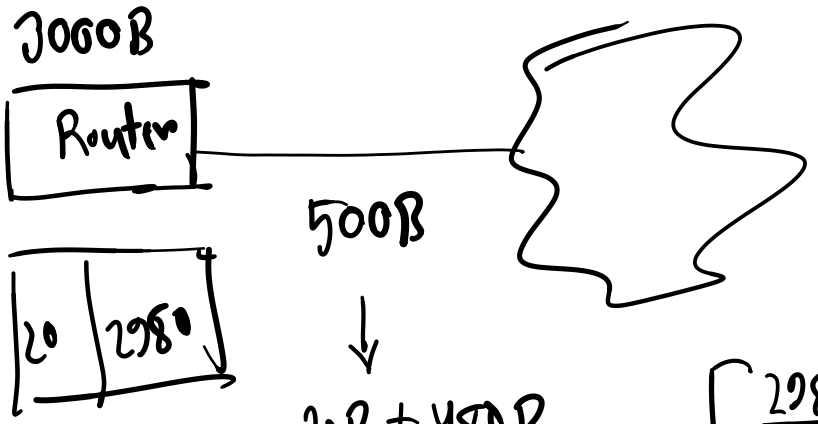
> Identification, Flags, Fragment Offset.

Why is fragment offset in units of 8 bytes?

> To save header space.

A datagram of 3000 B (20 B of IP header + 2980 B IP Payload) reached at Route and must be forward to link with MTU of 500 B. How many fragments will be generated and also write MF, offset, Total length value for all.

Soln:



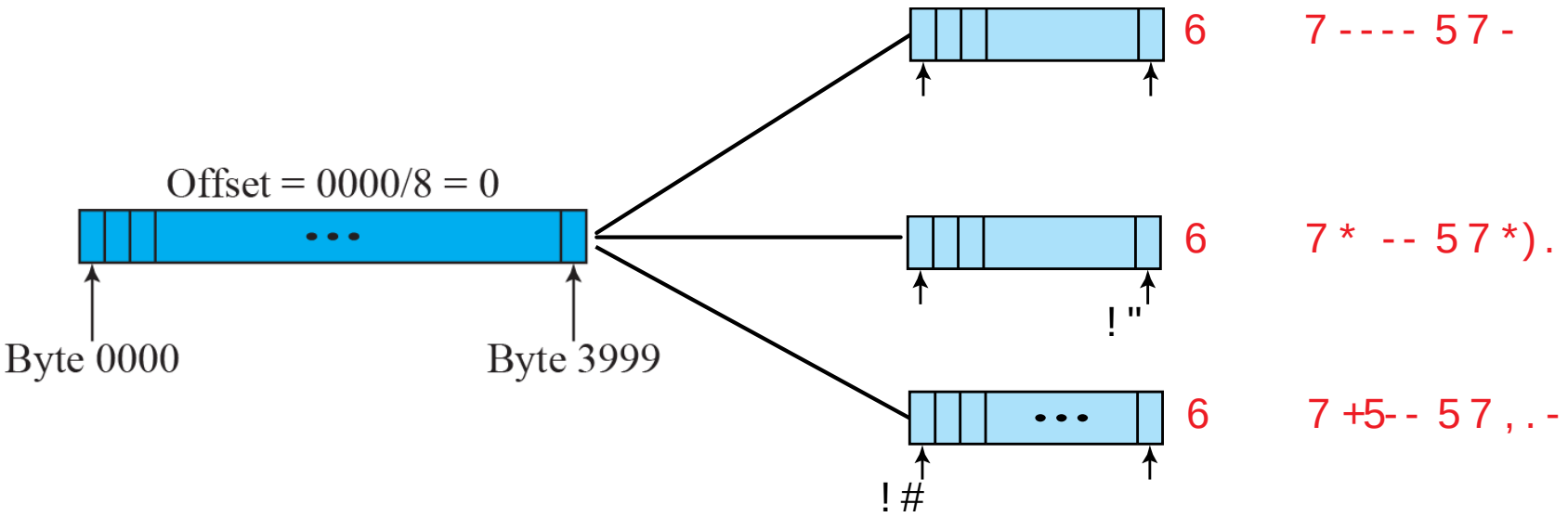
$\left\lceil \frac{2980}{480} \right\rceil = 7$ → fragments

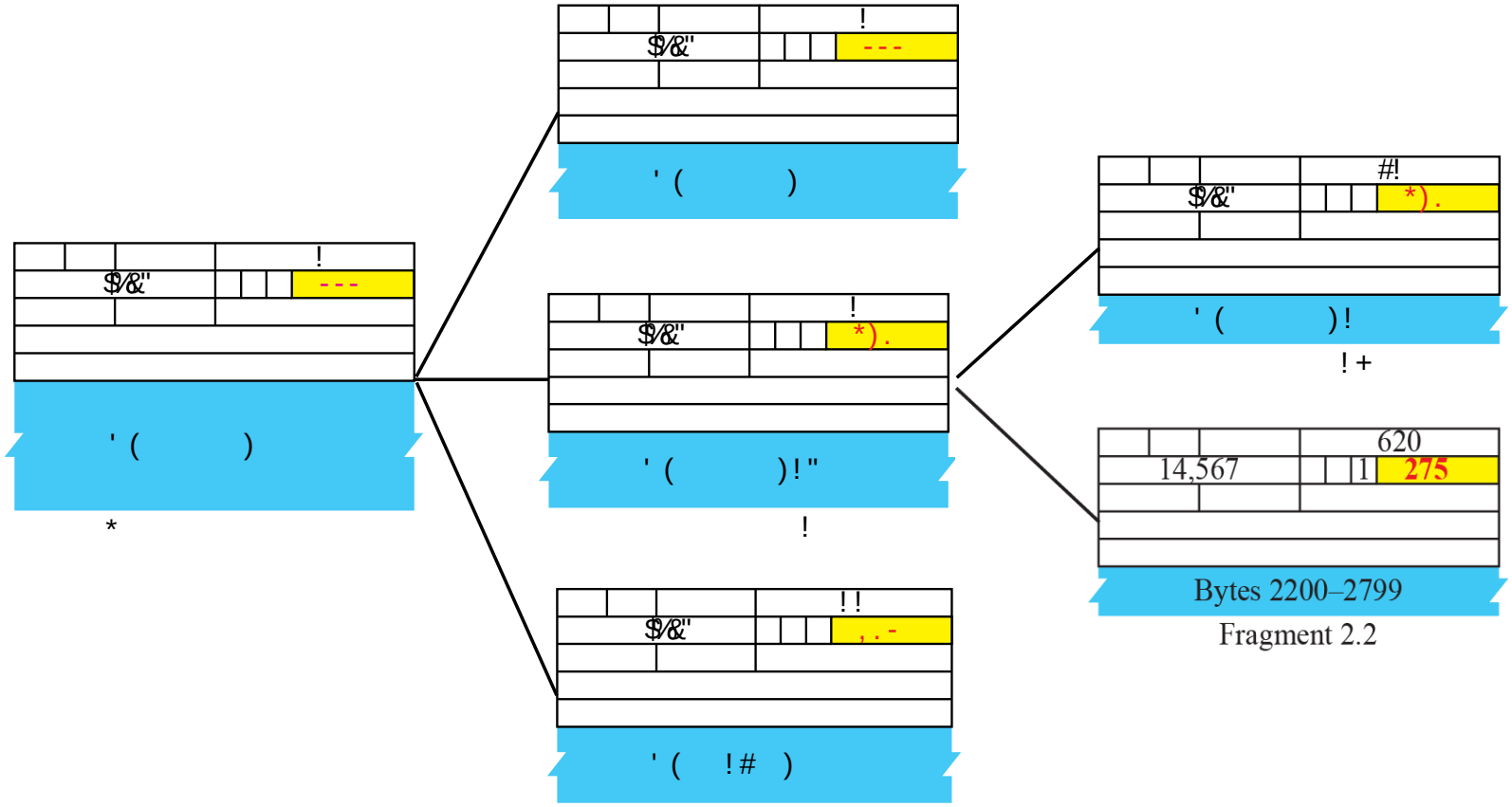
$\frac{480}{8} = 60$

$2980 - (480 \times 6) = 100$

P7	P6	P5	P4	P3	P2	P1	
100+20	480+20	480+20	480+20	480+20	480+20	480+20	
120	500	500	500	500	500	500	
0	1	1	1	1	1	1	MF
360	360	240	180	120	60	0	offset

D: Do not fragment
M: More fragments


Offset = 0000/8 = 0
Byte 0000
Byte 3999
6
6
6



Fragment 2.2

A packet has arrived with an M bit value of 0. Is this the first fragment, the last fragment, or a middle fragment? Do we know if the packet was fragmented?

If the M bit is 0, it means that there are no more fragments; the fragment is the last one. However, we cannot say if the original packet was fragmented or not. A nonfragmented packet is considered the last fragment.

A packet has arrived with an M bit value of 1. Is this the first fragment, the last fragment, or a middle fragment? Do we know if the packet was fragmented?

If the M bit is 1, it means that there is at least one more fragment. This fragment can be the first one or a middle one, but not the last one. We don t know if it is the first one or a middle one; we need more information (the value of the fragmentation offset). See also the next example.

A packet has arrived with an M bit value of 1 and a fragmentation offset value of zero. Is this the first fragment, the last fragment, or a middle fragment?

Because the M bit is 1, it is either the first fragment or a middle one. Because the offset value is 0, it is the first fragment.

A packet has arrived in which the offset value is 100. What is the number of the first byte? Do we know the number of the last byte?

To find the number of the first byte, we multiply the offset value by 8. This means that the first byte number is 800. We cannot determine the number of the last byte unless we know the length of the data.

A packet has arrived in which the offset value is 100, the value of HLEN is 5 and the value of the total length field is 100. What is the number of the first byte and the last byte?

The first byte number is 100 × 8 = 800. The total length is 100 bytes and the header length is 20 bytes (5 × 4), which means that there are 80 bytes in this datagram. If the first byte number is 800, the last byte number must be 879.

The header of the IP datagram is made of two parts: a fixed part and a variable part. The fixed part is 20 bytes long and was discussed in the previous section. The variable part comprises the options, which can be a maximum of 40 bytes.

Options, as the name implies, are not required for a datagram. They can be used for network testing and debugging. Although options are not a required part of the IP header, option processing is required of the IP software.

(
6

📦 IPv4 Options — Explained Clearly

IPv4 options are optional fields in the IP header used for control, debugging, and routing experiments.
They are rarely used today due to performance, security, and processing overhead.

📌 Maximum IP header size = 60 bytes
📌 Base header = 20 bytes
➡️ Maximum options size = 40 bytes

🔹 Categories of IPv4 Options

There are 6 commonly used options, divided into:

1️⃣ Single-Byte Options

(no Length or Data field)

Option	Type Value	Purpose
End of Option List	0	Marks end of options
No Operation (NOP)	1	Padding / alignment

2️⃣ Multiple-Byte Options

(require Type + Length + Data)

Option	Type	Purpose
Record Route	7	Records routers visited
Strict Source Route	137	Fixed route, no deviation
Loose Source Route	131	Route with flexibility
Timestamp	68	Records timestamps (and optionally IPs)

🔹 1️⃣ No-Operation (NOP) Option

✔ Purpose

- Used as a 1-byte filler
- Aligns the next option on 16-bit or 32-bit boundaries

✔ Format

makefile — Copy code

```
Type = 1
Binary: 00000001
```

✔ Usage Example

- Inserted between options
- Can appear multiple times

📌 Think of NOP as padding between options

🔹 2️⃣ End-of-Option List (EOL)

✔ Purpose

- Marks the end of the option field
- Tells receiver: *"Options are finished; payload begins"*

✔ Format

makefile — Copy code

```
Type = 0
Binary: 00000000
```

✔ Rules

- Can appear only once
- Must be the last option
- If alignment is needed → use NOP(s) before EOL

🔹 3️⃣ Record Route Option

✔ Purpose

- Records IP addresses of routers the packet passes through
- Used for debugging and path discovery

✔ Maximum routers recorded

- 9 routers max
- Due to 40-byte option size limit

✔ Format

Field	Description
Type = 7	Identifies option
Length	Total option size
Pointer	Points to next empty slot
IP Addresses	Filled by routers

📌 Pointer starts at 4 (first IP address field)

✔ How It Works

1. Source sends packet with empty address fields
2. Each router:
   - Inserts its outgoing interface IP
   - Increments pointer by 4
3. Stops when option is full

📌 Routers do NOT overwrite existing entries

🔹 4️⃣ Strict Source Route (SSR)

✔ Purpose

- Sender forces an exact route
- Packet must go only through listed routers

📌 If any unlisted router is encountered → ❌ packet dropped

✔ Type

```ini
Type = 137 (10001001)
```

✔ Format

Same as Record Route, BUT:

- IP addresses are pre-filled by sender
- No empty slots

✔ Router Behavior

- Router checks:
  - Incoming interface IP == expected IP?
- If yes:
  - Swaps destination address
  - Advances pointer
- If no:
  - Packet discarded + ICMP error

📌 Very strict and fragile

🔹 5️⃣ Loose Source Route (LSR)

✔ Purpose

- Similar to SSR, but relaxed
- Packet must visit listed routers
- May pass through other routers too

✔ Type

```ini
Type = 131 (10000011)
```

📌 More practical than strict source routing
📌 Still rarely used today

🔹 6️⃣ Timestamp Option

✔ Purpose

- Records time (ms since midnight UTC) at routers
- Used for:
  - Delay estimation
  - Network diagnostics

✔ Type

```ini
Type = 68 (01000100)
```

✔ Fields Explained

Field	Meaning
Pointer	Next available slot
Overflow	Count of routers unable to add timestamp
Flags	Controls behavior
IP + Timestamp	Data recorded

✔ Flag Meanings

Flag	Behavior
0	Routers add timestamp only
1	Routers add IP + timestamp
3	IPs pre-filled, routers match & timestamp

📌 Overflow increments if space runs out

🔴 Why IPv4 Options Are Rarely Used

❌ Slow processing (routers must inspect header)
❌ Security risks (source routing attacks)
❌ Most routers drop or ignore options
❌ Modern tools (Traceroute, ICMP, MPLS) replaced them

🧠 Exam-Ready Summary Table

Option	Type	Size	Main Use
End of Option	0	1 byte	End marker
No Operation	1	1 byte	Padding
Record Route	7	Variable	Path tracing
Strict Source Route	137	Variable	Forced routing
Loose Source Route	131	Variable	Semi-forced routing
Timestamp	68	Variable	Delay measurement

Figure 7.10 "

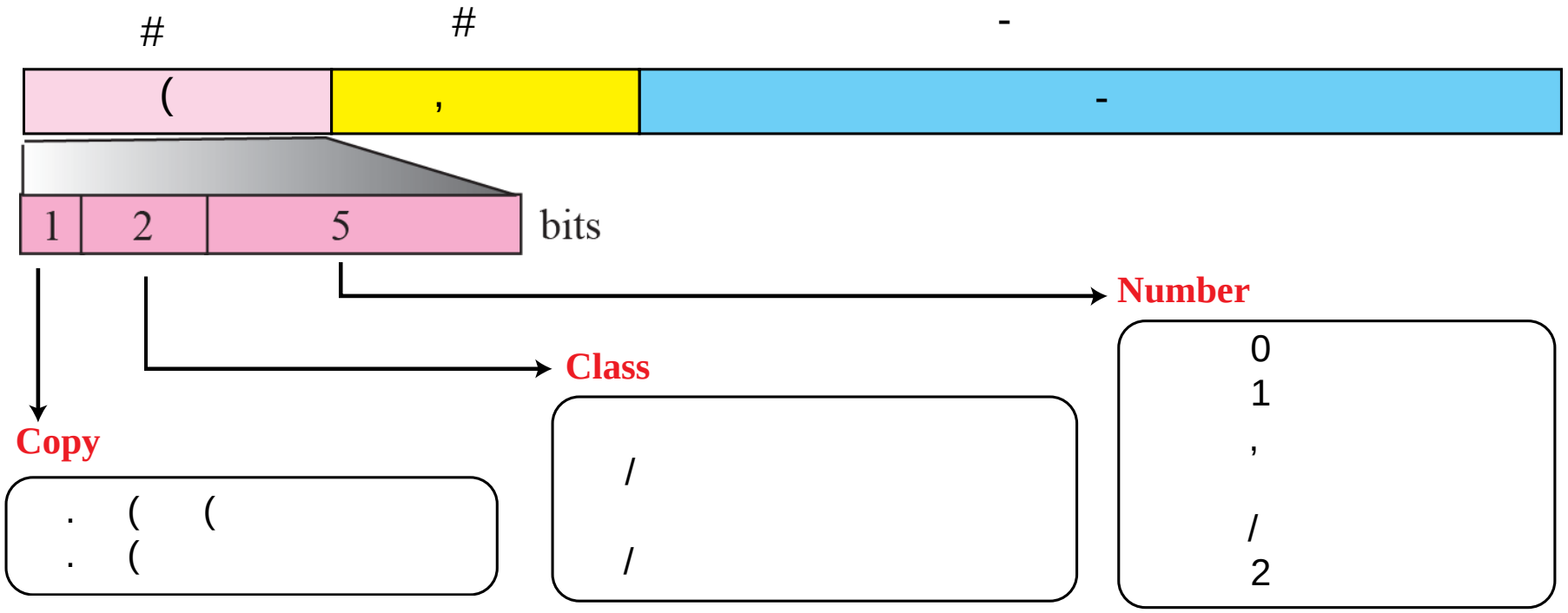


Figure 7.11

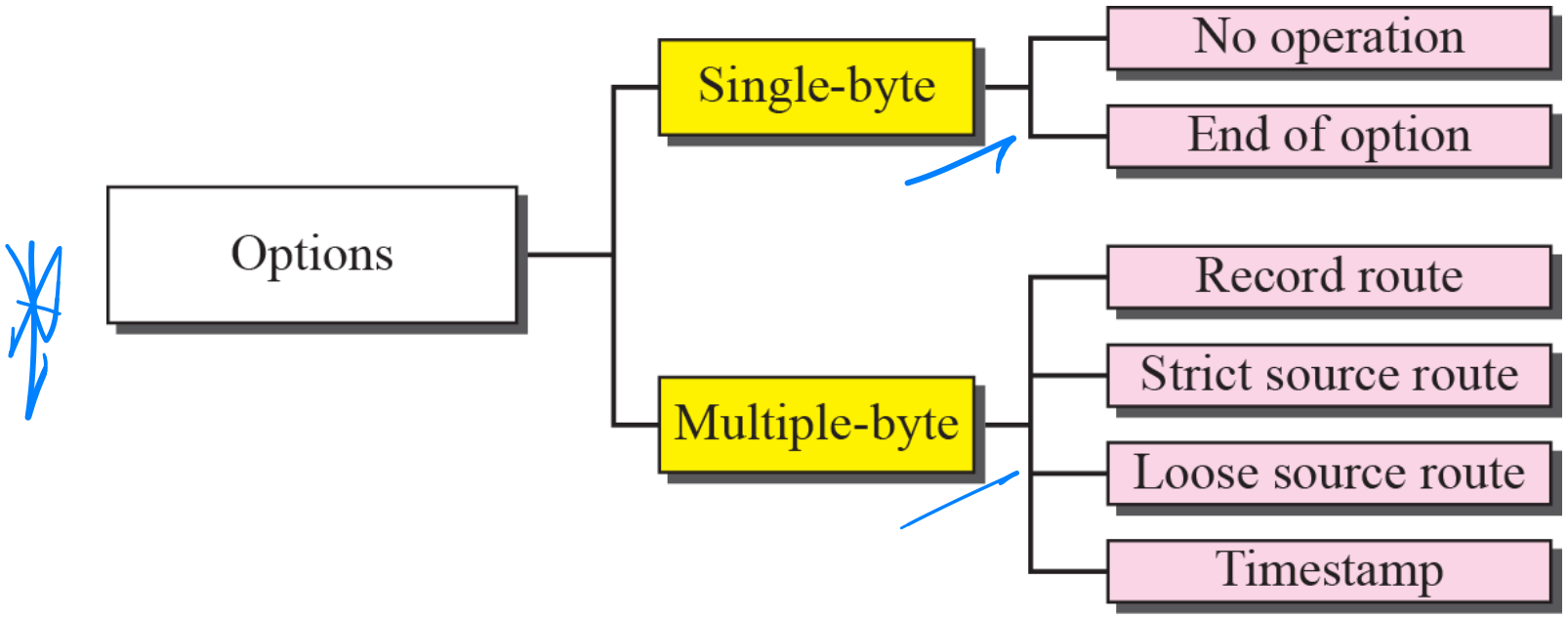
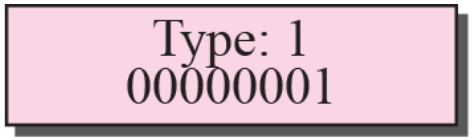
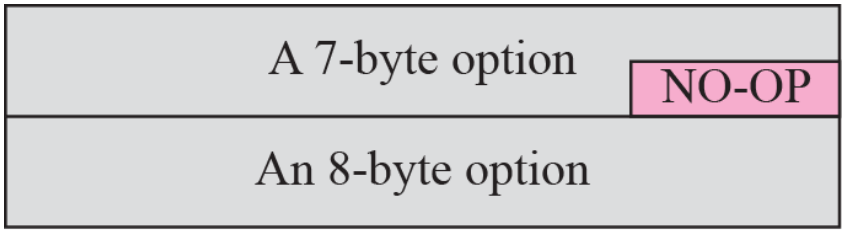

Options
Single-byte
No operation
End of option
Multiple-byte
Record route
Strict source route
Loose source route
Timestamp

Figure 7.12



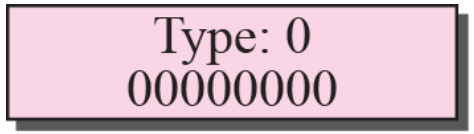
a. No operation option

b. Used to align beginning of an option

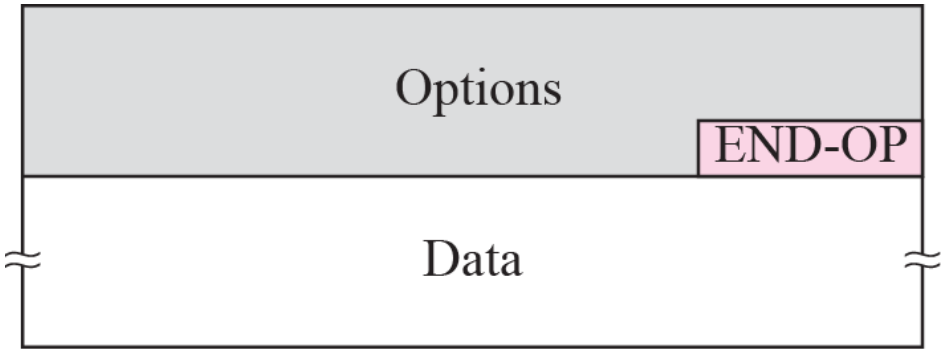


c. Used to align the next option

Figure 7.13 #



a. End of option



b. Used for padding

Figure 7.14 $

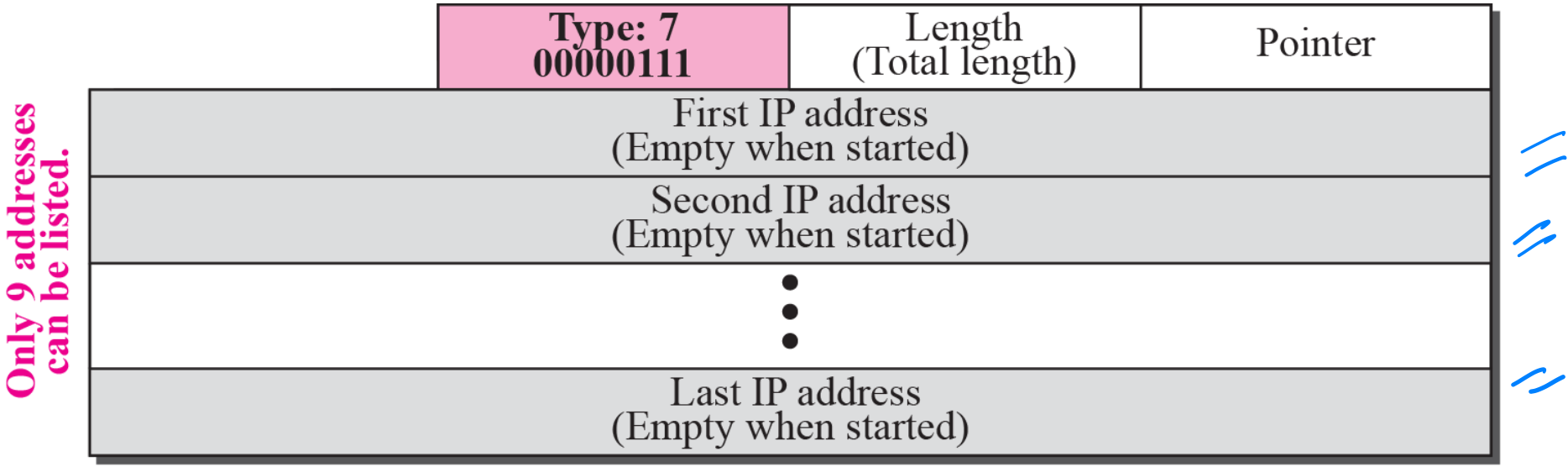

Type: 7
00000111
Length
(Total length)
Pointer
First IP address
(Empty when started)
Second IP address
(Empty when started)
Last IP address
(Empty when started)
Only 9 addresses can be listed.

Figure 7.15 $

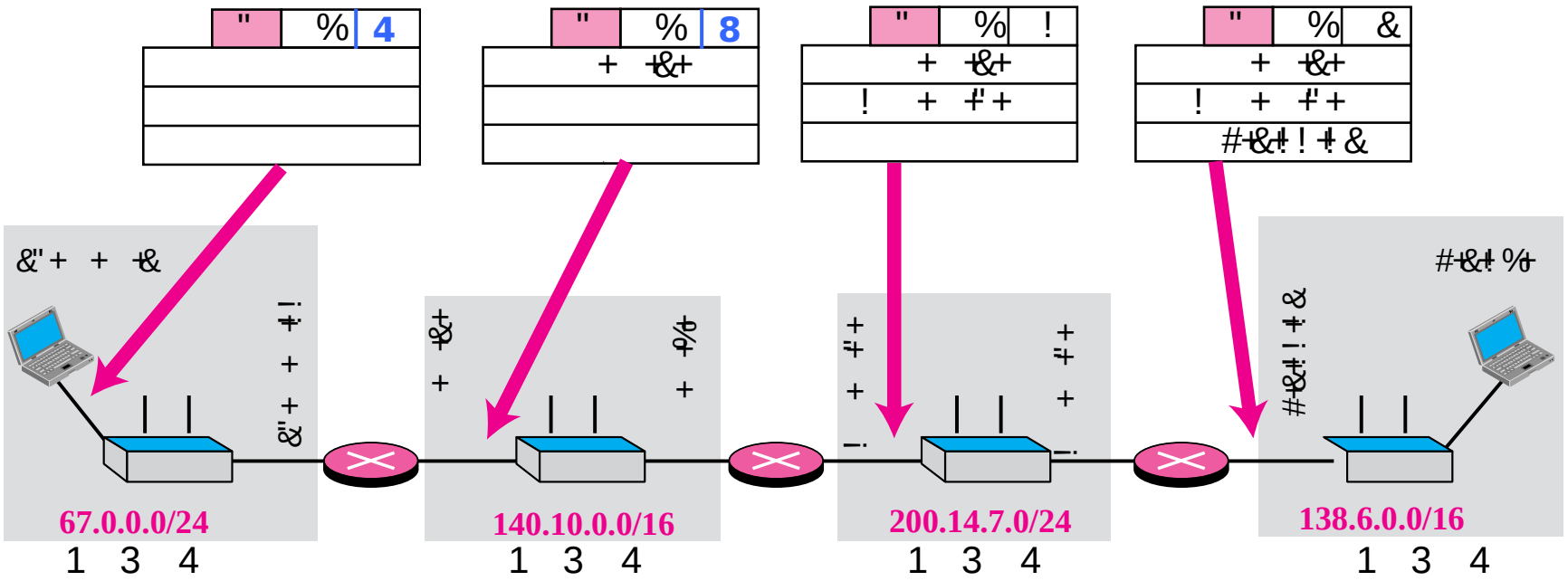


Figure 7.16 #

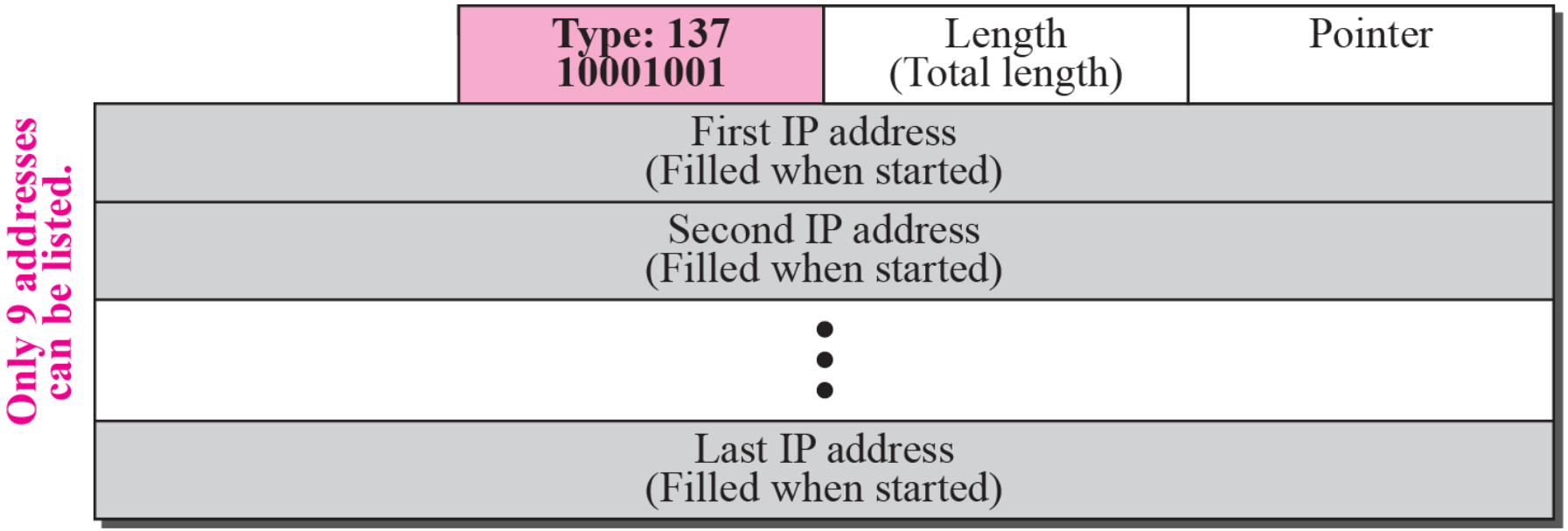

Type: 137
10001001
Length
(Total length)
Pointer
First IP address
(Filled when started)
Second IP address
(Filled when started)
Last IP address
(Filled when started)
Only 9 addresses can be listed.

Figure 7.17 #

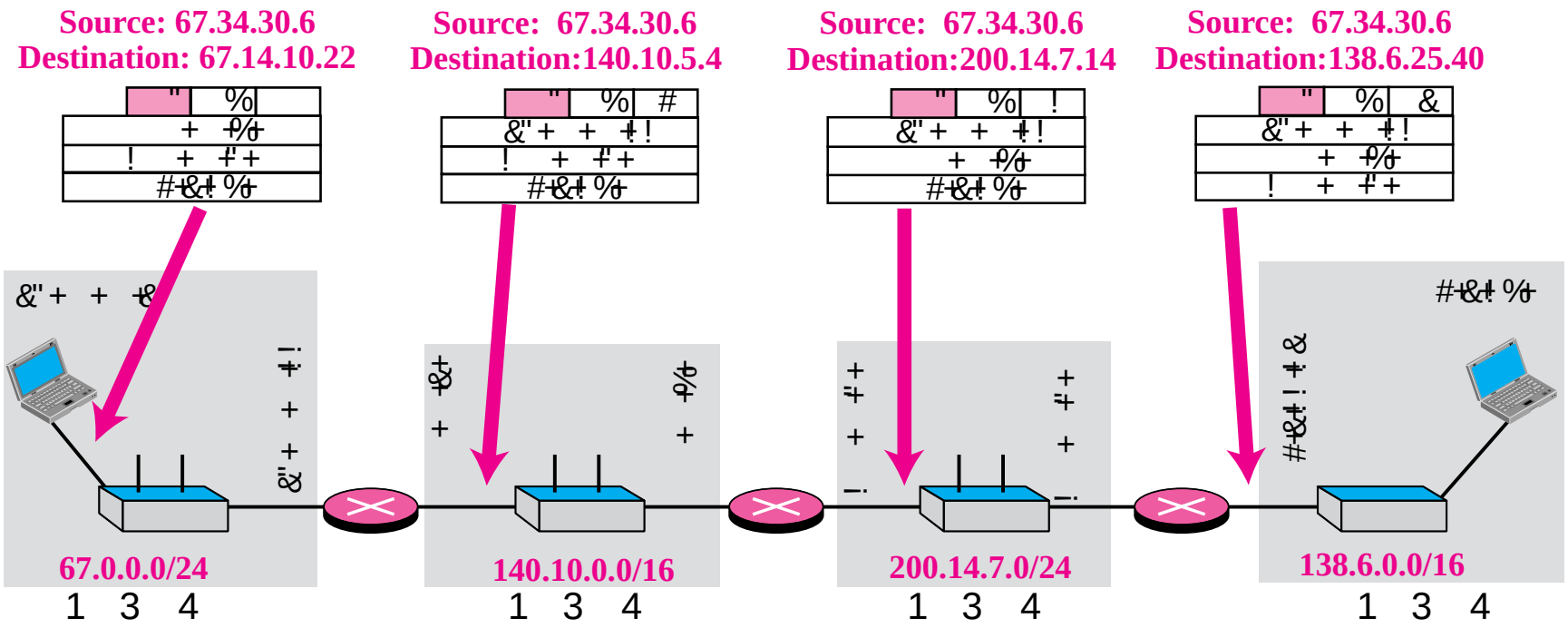

Source: 67.34.30.6
Destination: 67.14.10.22
Source: 67.34.30.6
Destination:140.10.5.4
Source: 67.34.30.6
Destination:200.14.7.14
Source: 67.34.30.6
Destination:138.6.25.40
67.0.0.0/24
140.10.0.0/16
200.14.7.0/24
138.6.0.0/16
1 3 4
1 3 4
1 3 4
1 3 4

Figure 7.18 % # #

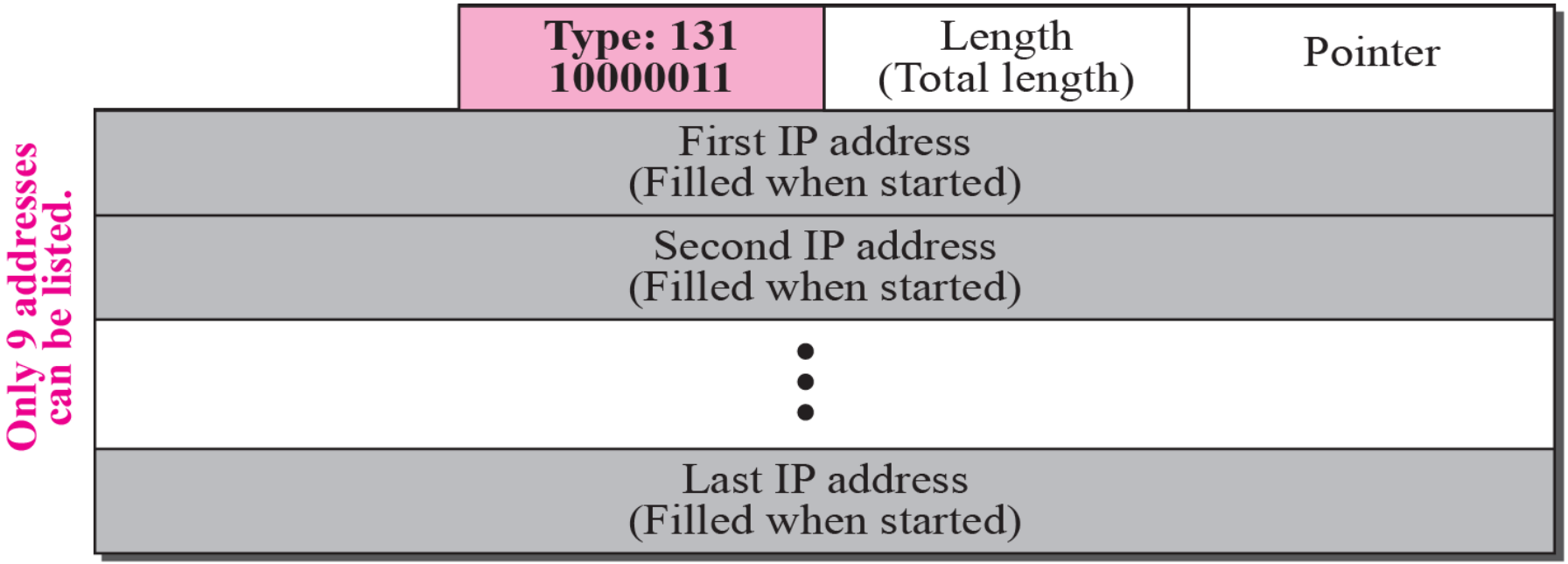

Type: 131
10000011
Length
(Total length)
Pointer
First IP address
(Filled when started)
Second IP address
(Filled when started)
Last IP address
(Filled when started)
Only 9 addresses can be listed.

Figure 7.19

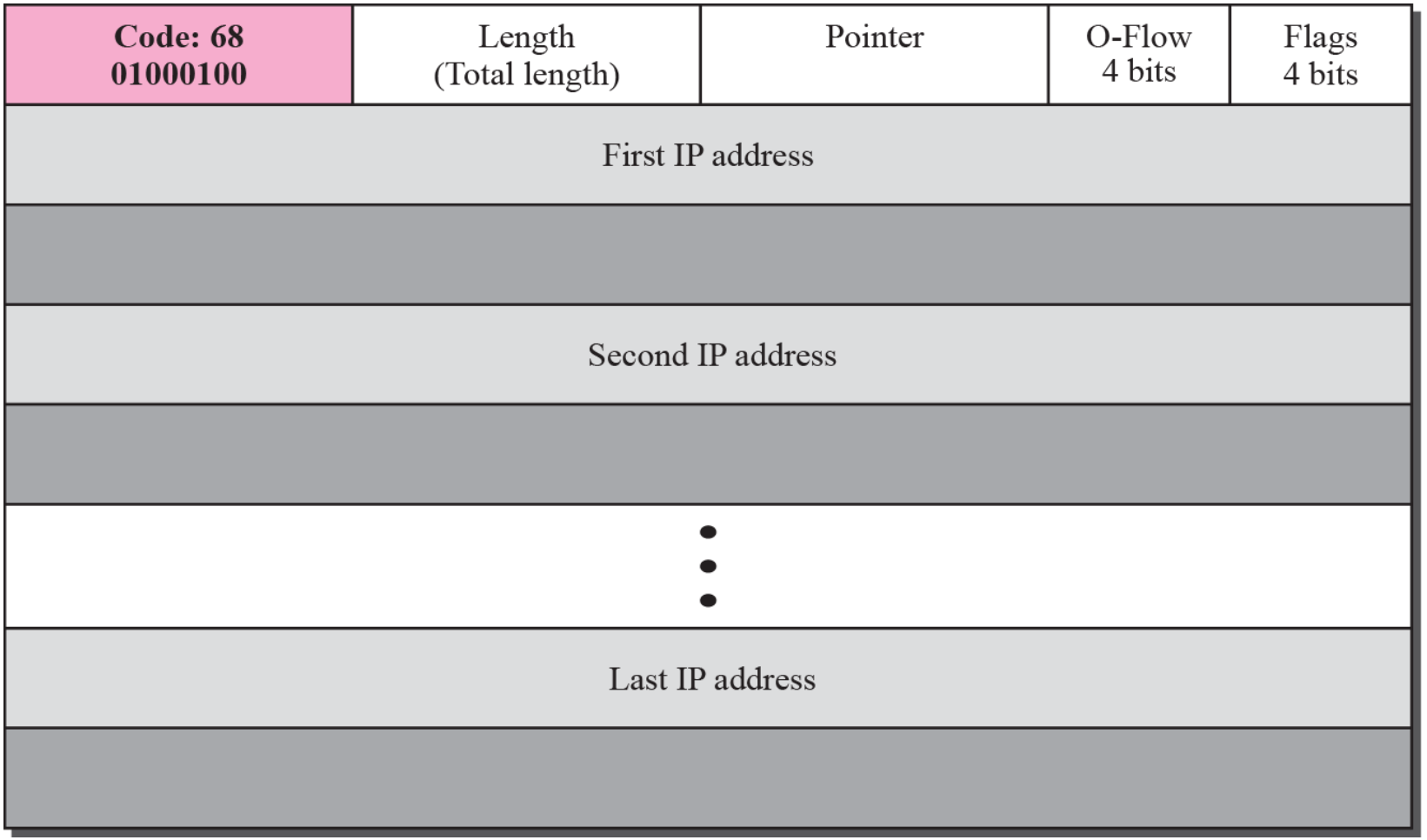

Code: 68
01000100
Length
(Total length)
Pointer
O-Flow
4 bits
Flags
4 bits
First IP address
Second IP address
Last IP address

Figure 7.20

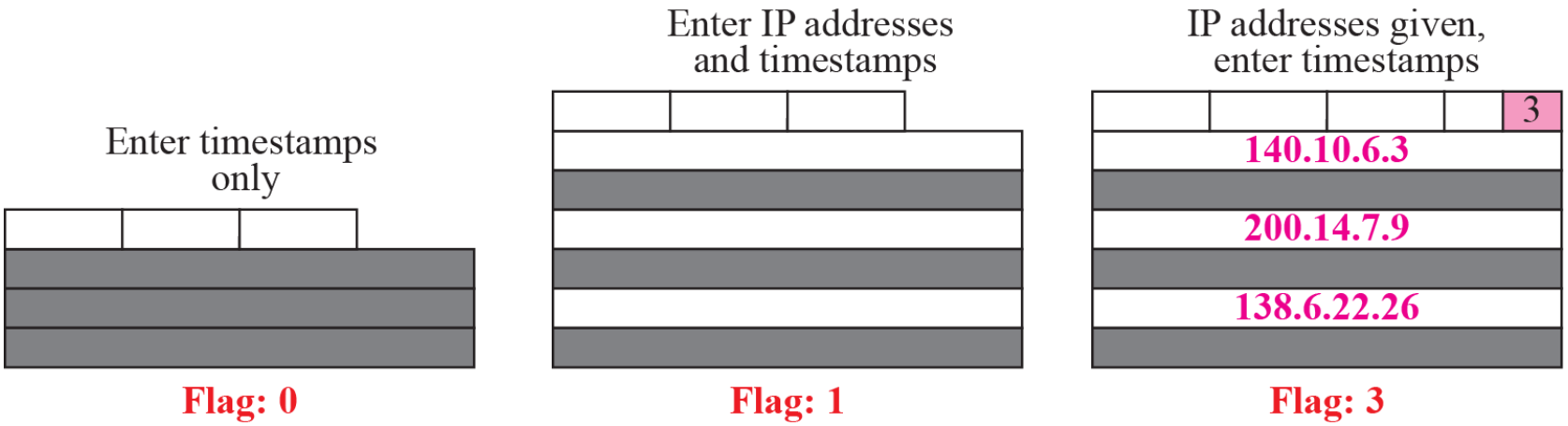
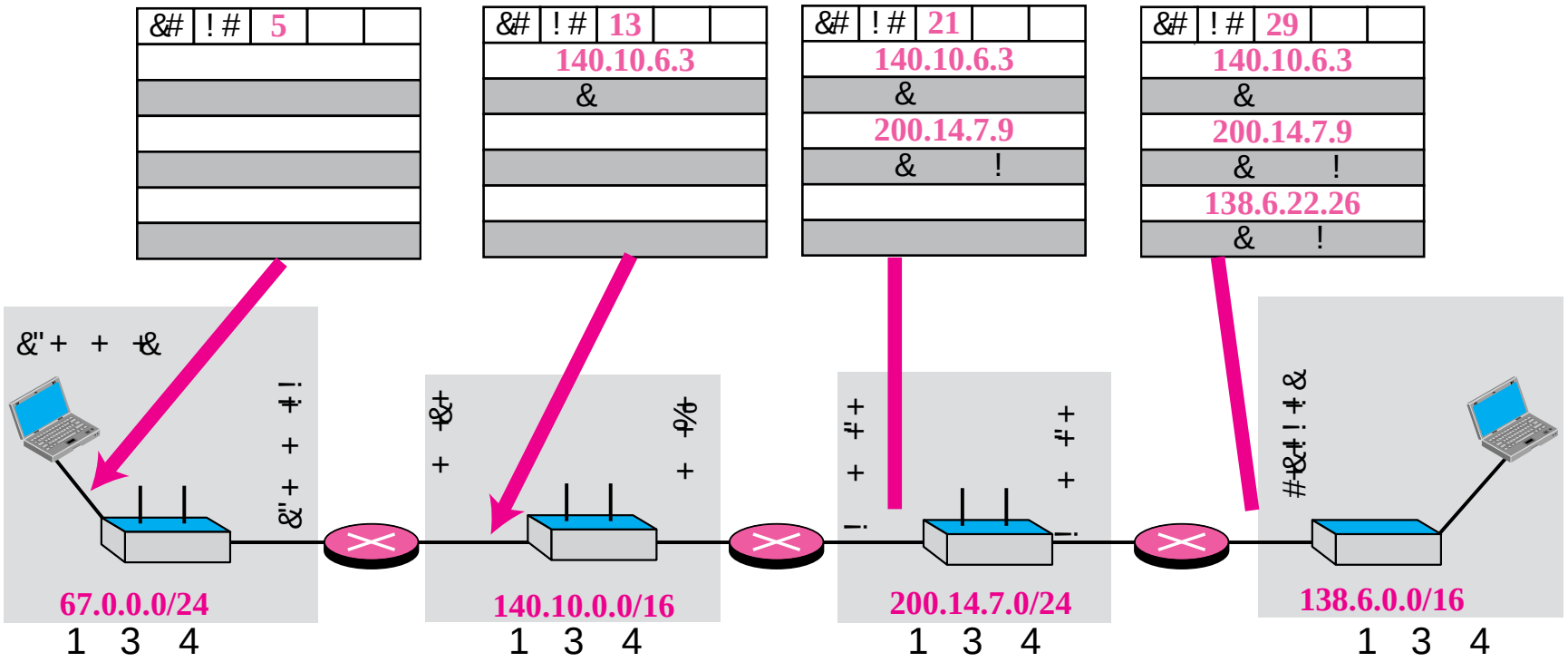

Enter timestamps only
Flag: 0
Enter IP addresses and timestamps
Flag: 1
IP addresses given, enter timestamps
3
140.10.6.3
200.14.7.9
138.6.22.26
Flag: 3

Figure 7.21


&# ! # 5
&# ! # 13
140.10.6.3
&
&# ! # 21
140.10.6.3
&
200.14.7.9
& !
&# ! # 29
140.10.6.3
&
200.14.7.9
& !
138.6.22.26
& !
67.0.0.0/24
1 3 4
140.10.0.0/16
1 3 4
200.14.7.0/24
1 3 4
138.6.0.0/16
1 3 4

Example 7.10

Which of the six options must be copied to each fragment?

We look at the first (left-most) bit of the type for each option.

a. No operation: type is 00000001; not copied.
b. End of option: type is 00000000; not copied.
c. Record route: type is 00000111; not copied.
d. Strict source route: type is 10001001; copied.
e. Loose source route: type is 10000011; copied.
f. Timestamp: type is 01000100; not copied.

Example 7.11

Which of the six options are used for datagram control and which for debugging and managements?

We look at the second and third (left-most) bits of the type.

a. No operation: type is 00000001; datagram control.
b. End of option: type is 00000000; datagram control.
c. Record route: type is 00000111; datagram control.
d. Strict source route: type is 10001001; datagram control.
e. Loose source route: type is 10000011; datagram control.
f. Timestamp: type is 01000100; debugging and management control.

Example 7.12

One of the utilities available in UNIX to check the traveling of the IP packets is ping. In the next chapter, we talk about the ping program in more detail. In this example, we want to show how to use the program to see if a host is available. We ping a server at De Anza College named fhda.edu. The result shows that the IP address of the host is 153.18.8.1. The result also shows the number of bytes used.

```
$  ping fhda.edu
PING fhda.edu (153.18.8.1) 56(84) bytes of data.
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq =
0 ttl=62 time=1.87 ms
...
```

Example 7.13

We can also use the ping utility with the -R option to implement the record route option. The result shows the interfaces and IP addresses.

```
$ ping -R fhda.edu
PING fhda.edu (153.18.8.1) 56(124) bytes of data.
64 bytes from tiptoe.fhda.edu
(153.18.8.1): icmp_seq=0 ttl=62 time=2.70 ms
RR:  voyager.deanza.fhda.edu (153.18.17.11)
     Dcore_G0_3-69.fhda.edu (153.18.251.3)
     Dbackup_V13.fhda.edu (153.18.191.249)
     tiptoe.fhda.edu (153.18.8.1)
     Dbackup_V62.fhda.edu (153.18.251.34)
     Dcore_G0_1-6.fhda.edu (153.18.31.254)
     voyager.deanza.fhda.edu (153.18.17.11)
```

Example 7.14

The traceroute utility can also be used to keep track of the route of a packet. The result shows the three routers visited.

```
$ traceroute fhda.edu
traceroute to fhda.edu (153.18.8.1), 30 hops max, 38 byte packets
 1 Dcore_G0_1-6.fhda.edu (153.18.31.254)  0.972 ms  0.902 ms
   0.881 ms
 2 Dbackup_V69.fhda.edu (153.18.251.4)  2.113 ms  1.996 ms
   2.059 ms
 3 tiptoe.fhda.edu (153.18.8.1)  1.791 ms  1.741 ms  1.751 ms
```

Example 7.15

The traceroute program can be used to implement loose source routing. The -g option allows us to define the routers to be visited, from the source to destination. The following shows how we can send a packet to the fhda.edu server with the requirement that the packet visit the router 153.18.251.4

```
$ traceroute -g  153.18.251.4 fhda.edu.
traceroute to fhda.edu (153.18.8.1), 30 hops max, 46 byte packets
 1  Dcore_G0_1-6.fhda.edu (153.18.31.254)  0.976 ms  0.906 ms
    0.889 ms
 2  Dbackup_V69.fhda.edu (153.18.251.4)  2.168 ms  2.148 ms
    2.037 ms
```

Example 7.16

The traceroute program can also be used to implement strict source routing. The -G option forces the packet to visit the routers defined in the command line. The following shows how we can send a packet to the fhda.edu server and force the packet to visit only the router 153.18.251.4.

```
$ traceroute -G  153.18.251.4 fhda.edu.
traceroute to fhda.edu (153.18.8.1), 30 hops max, 46 byte packets
  1  Dbackup_V69.fhda.edu (153.18.251.4)  2.168 ms  2.148 ms
     2.037 ms
```

' (&)# "

The error detection method used by most TCP/IP protocols is called the checksum. The checksum protects against the corruption that may occur during the transmission of a packet. It is redundant information added to the packet. The checksum is calculated at the sender and the value obtained is sent with the packet. The receiver repeats the same calculation on the whole packet including the checksum. If the result is satisfactory (see below), the packet is accepted; otherwise, it is rejected.

Checksum Calculation at the Sender

Checksum Calculation at the Receiver

Checksum in the Packet

Figure 7.22 &

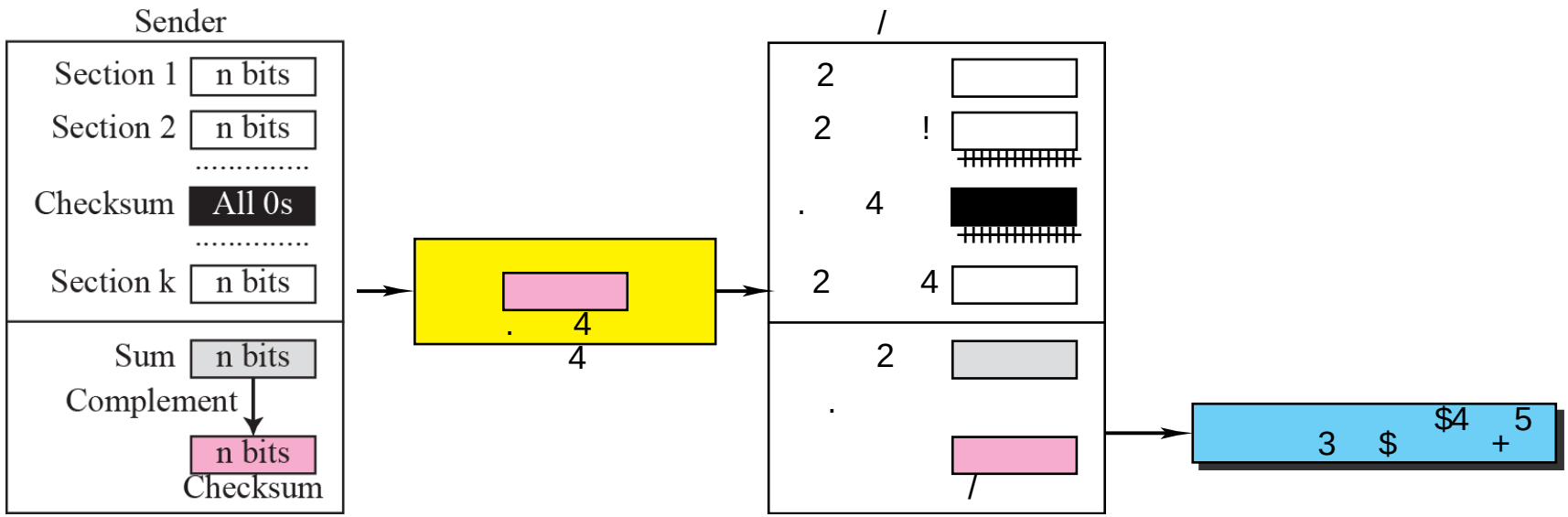

Sender
Section 1
n bits
Section 2
n bits
Checksum
All 0s
Section k
n bits
Sum
n bits
Complement
n bits
Checksum

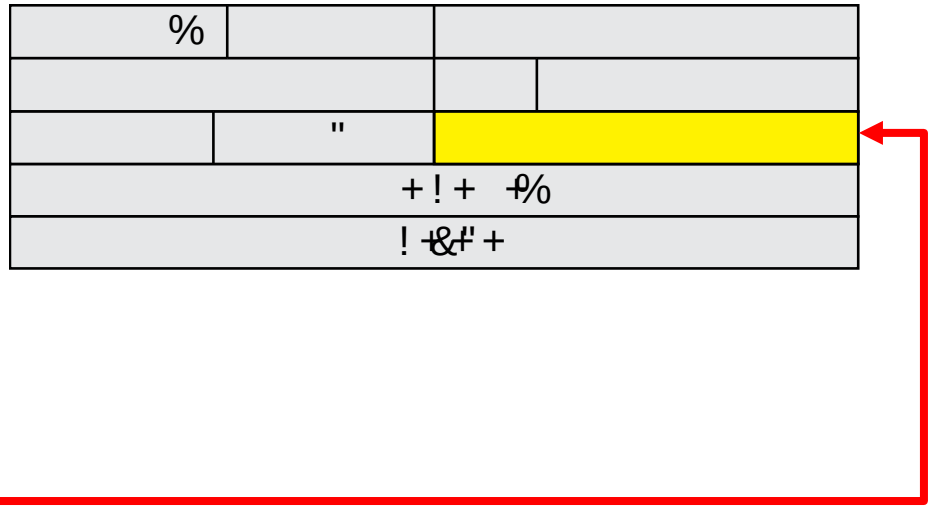
Figure 7.23 & '

Example 7.17

Figure 7.24 shows an example of a checksum calculation at the sender site for an IP header without options. The header is divided into 16-bit sections. All the sections are added and the sum is complemented. The result is inserted in the checksum field.

Figure 7.24 &

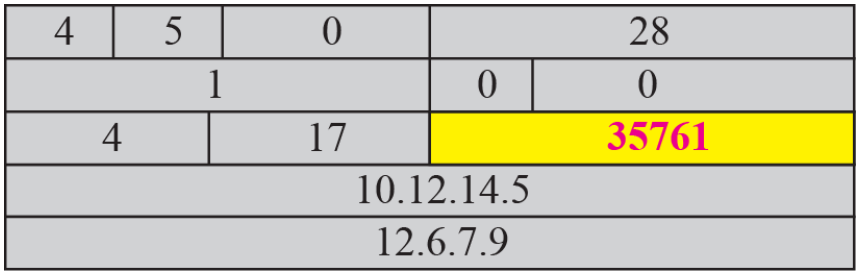
4, 5, and 0	01000101	00000000
28	00000000	00011100
1	00000000	00000001
0 and 0	00000000	00000000
4 and 17	00000100	00010001
0	00000000	00000000
10.12	00001010	00001100
14.5	00001110	00000101
12.6	00001100	00000110
7.9	00000111	00001001
Sum	**01110100**	**01001110**
Checksum	**10001011**	**10110001**



Example 7.18

Figure 7.25 shows the checking of checksum calculation at the receiver site (or intermediate router) assuming that no errors occurred in the header. The header is divided into 16-bit sections. All the sections are added and the sum is complemented. Since the result is 16 0s, the packet is accepted.

Figure 7.25 &



4, 5, and 0	→	01000101 00000000
28	→	00000000 00011100
1	→	00000000 00000001
0 and 0	→	00000000 00000000
4 and 17	→	00000100 00010001
Checksum	→	**10001011 10110001**
10.12	→	00001010 00001100
14.5	→	00001110 00000101
12.6	→	00001100 00000110
7.9	→	00000111 00001001
Sum	→	**1111 1111 1111 1111**
Checksum	→	**0000 0000 0000 0000**